Modules at scale

Document number: P0841R0
Audience: EWG
Bruno Cardoso Lopes, *Adrian Prantl*, *Duncan P. N. Exon Smith*
2017-10-16

This paper discusses Apple's experience with modules on its software ecosystem and how the current state of the Modules TS lacks the necessary expressiveness to reflect Apple's requirements.

Background

Apple currently uses Clang Modules, a *Modules* design and implementation that's specific to clang and contains support for different C-based languages; C, Objective-C, Objective-C++ and C++. The intent of this paper is not to sell *Clang Modules* but to provide insights based on the current requirements of Apple's libraries. Ideally we would like to incorporate changes into the Modules TS that would allow Apple to support C++ Modules in its shipping library interface.

Modules at Apple

Clang's modules implementation predates the Modules TS and was first introduced in 2012. Since then Apple has been shipping modularized library interfaces (headers) in its SDKs, enabled the use of modules by default for all new macOS/iOS projects, and facilitated the widespread use of modules by third-party libraries on Apple platforms. We also support modules internally for hundreds of internal projects.

The majority of these projects are written in Objective-C or Objective-C++, and although not strictly C++, they export C++ or C++-compatible interfaces, which are consumed by internal or external C++ projects; Clang and Webkit are examples of such projects.

Interoperability through modules

The *Swift* language supports interoperation with C and Objective-C, which relies on Modules for bridging the languages. Apple's developer tools provide transparent generation of modules for C and Objective-C libraries, meaning that the majority of Swift developers targeting Apple's platforms indirectly use Modules, because:

- Swift access to Apple APIs is done through the C and Objective-C headers coming from SDKs.
- The primary way to access third party library APIs written in C and Objective-C in Swift is through a modular interface.

Header Modules

To better explain Apple's use of *Clang Modules* in Modules TS terms, Apple ships and provides customers the equivalent of the *module interface unit* part of a library, meaning that Apple provides one module for each library, where the exported content comes from a set of headers. The set of headers in a library is entirely described in one module, which may or may not contain associated submodules.

Different libraries in Apple's SDKs contain dependencies between each other. Apple's SDK is almost entirely modularized bottom-up, meaning that the relationship between all library dependencies is a chain of module

imports and few non-modular includes. This allows customers to consume modules at almost every single point where a header from the SDK is used.

Modularization of headers are described in a special textual file (*modulemap*) that maps headers to *modules* and *submodules*. Clang performs a module import transparently: parsing a `#include` directive triggers a module import if a *modulemap* is found for the header.

Export control is done at module granularity level, for each module one can specify if the entire module, a few selected submodules or nothing at all is exported - see Clang Modules documentation for details.

For instance, two possible ways of mapping are:

1. A main header (*umbrella header*) that includes all other headers that are part of the library interface. Each header in the umbrella is automatically assigned to a submodule.
2. The explicit list of headers that are part of the module, with possible inline declaration of submodules and their sub-sequential explicit list of headers or an umbrella header.

To illustrate, take library `Foo` with umbrella header `Foo.h`, including `FooA.h` and `FooB.h` headers. This corresponds to module `Foo` with submodules `Foo.FooA` and `Foo.FooB`.

In order to properly modularize bottom-up, Apple ships *modulemaps* in its standard C++ library (libcxx) and C library (Darwin's `PATH_TO_SDK/usr/include`).

Problems with the current form of Modules TS

The Modules TS (10.7.2) states that there can only be one *module interface unit* file in a module:

> A module interface unit is a module unit whose module-declaration contains the export keyword; any other module unit is a module implementation unit. A named module shall contain exactly one module interface unit.

This is a showstopper for us, because as mentioned in the previous section, Apple's libraries consists of one or more headers, where any of them can contribute content to be exported in the final module. Additionally, the headers are shared among other supported C-based languages, and we strongly believe it's a reasonable model to continue shipping.

Similar concerns have been mentioned before, see the *Module partitions* section from P0273R0.

Macros

Given that Apple library interfaces support different C-based languages, we rely on macros in order to correctly gather availability information, heterogeneous platform support and to reason on top of features. Because Apple's SDK is entirely modularized, these macros are available for consumption at any library interface level, being critical in Apple's chain of module imports.

For example, as one can see in `LinearAlgebra/base.h` shipped with the SDK in macOS 10.13, the header use macros to control availability information for a library:

```
/*  Define abstractions for a number of attributes that we wish to be able to
concisely attach to functions in the LinearAlgebra library.            */
#define LA_AVAILABILITY  __OSX_AVAILABLE_STARTING(__MAC_10_10,__IPHONE_8_0)
...
```

```
#define LA_FUNCTION      OS_EXPORT OS_NOTHROW
#define LA_CONST         OS_CONST
```

Note that `__MAC_10_10` is available through another module that provides macros from `Availability.h`. Apple's customers depend on macros with modules like these.

Problems with the lack of Macros in Modules TS

The Modules TS lacks support for macros. According to Section 3.2 in p0142r0:

> ... because the preprocessor is largely independent of the core language, it is impossible for a tool to understand (even grammatically) source code in header files without knowing the set of macros and configurations that a source file including the header file will activate. It is regrettably far too easy and far too common to under-appreciate how much macros are (and have been) stifling development of semantics-aware programming tools and how much of drag they constitute for C++, compared to alternatives...

Using Modules as a way to break away from macros seems fair but we believe a transition path is necessary, since no support for macros blocks Apple's path to adopting C++ Modules. Additionally, concerns from others were already outlined in P0273R1.

We propose support for macros with the intent of helping with migration. To achieve it, we suggest adding extra syntax that:

- Adds on top of existing Modules TS syntax.
- Can easily be later deprecated and removed, without polluting the reserved keyword namespace.

Proposed macros syntax

To export macros defined in a module, we propose augmenting the *module-declaration* in a *module interface unit* with a submodule-like specification:

```
export module M; // declare module M
...
export module M.__macros; // M export all macros
```

In the code snippet above, all macros in `M`'s *module interface unit* are exported. To select macros to export, a macro identifier can be explicitly specified with `export module M.__macros.<MACRO_NAME>` as if accessing a reserved submodule:

```
export module M.__macros.INFINITY; // M exports macro INFINITY
...
export module M.__macros.HUGE_VAL; // M exports macro HUGE_VAL
```

On the module consumer side, a similar mechanism is used. To import the complete set of macros exported by module `M`:

```
import M.__macros; // import all macros exported in module M
```

Similarly, for importing specific known macros from module `M`:

```
import M.__macros.INFINITY; // import macro INFINITY from module M
import M.__macros.HUGE_VAL; // import macro HUGE_VAL from module M
```

Note that exporting all macros in module `M` doesn't by default make the macros visible to importers of `M`. Importers must explicitly `import M._macros` or `import M._macros.<MACRO_NAME>` to make macros defined in `M` visible. This enforces judicious use of macros in a modular world.

The use of `import` for macros is only allowed *after* a *module-declaration* in the same file. Likewise `export` is only valid after a *module-import-declaration* in the same file.

Representing macros with syntactic sugar allows for deprecation of the mechanism later on, without pollution of the top-level reserved keyword space.

The `module` keyword

It is not clear in the Modules TS whether `module` is an always reserved or context-sensitive reserved keyword.

Apple has been shipping for several years a kernel extension interface in C++, where the identifier `module` is used to name members describing a kernel module, as one example in `/usr/include/mach/host_priv.h`:

```
...
kern_return_t kmod_create
(
    host_priv_t host_priv,
    vm_address_t info,
    kmod_t *module
);
```

For instance, some versions of Clang would complain and error out when including the code above because of the keyword conflict in Clang's implementation of the Modules TS.

We propose that `module` becomes a context-sensitive reserved keyword allowing existing libraries and and interfaces to continue using it in contexts where no semantic conflict with modules is possible.